

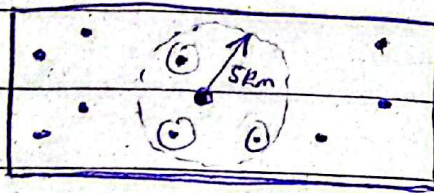
Date _____

Day _____

"BOOKING.COM"

— Search Engine —

→ Proximity Search → finding Nearby locations



→ Static locations

↳ Similar to Uber / Careem

↳ Diff is of Dynamic location → Captain
→ User

→ Functional Requirements

↳ Find nearby locations / places

↳ given $x, y = (\text{long}, \text{lat})$ by user
find "n" nearby places

↳ Filters → Rankings → Popularity

↳ Reviews → Avg ratings
→ Sponsored Ads

↳ ratings
↳ comments
↳ photos

↳ Locations (metadata)

→ Vendors can add/delete/update

↳ Photos } details

↳ it's location

↳ Description

Date _____

Day _____

→ Non-Functional Requirements

→ Response time : $P(95) = 150\text{ms}$

↳ Why fast resp time → ↑ user exp → ↑ conversion ↓
↑ profits

→ High Availability

→ 500 M locations listed

→ 100,000 searches/second ⇒ QPS

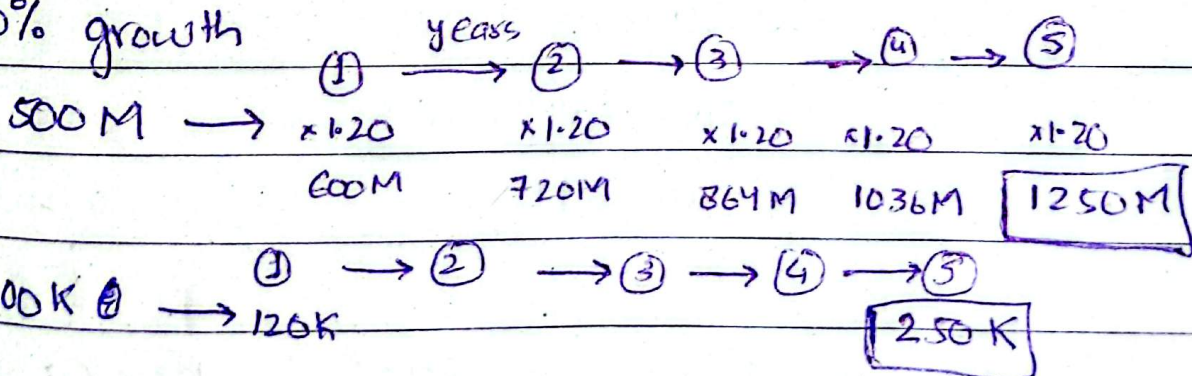
→ 20% growth rate

Step#01 : Storage Calculations

of locations = 500M

QPS = 100,000 QPS

20% growth



Date _____

Day _____

~~Step #~~

- Schema Design

Location Table

LocationId	:	8 B
Name	:	256 B
Longitude	:	8 B
Latitude	:	8 B
Description	:	512 B
Category (Enum)	:	1 B
= 793 B		
→ 1 Row		

793 B → 500 M

→ 500 M × 793 B

→ $500 \times 10^6 \times 793$

→ 400 GB

5 Years → $400 \text{ GB} \times 1.20^5 \approx 1000 \text{ GB}$

Date _____

Day _____

ReviewTable

	ReviewId	→ What would be Datatype?
	Text	Auto ✓ UUID
	Rating	Datetime ✓ (so we get sorted by locations)
Fk	Location	→ 1 Row = 525 B → Data Access pattern

→ Show all Reviews for location X

Clustered Index Key: (locationId, Rating ID)

$\pi_1 : R_1, R_2, R_3$
$\pi_2 : R_4, R_5, R_8$

- If there are frequent page splits it slows down Insertion operation

→ Covering Index

ON (Reviews ID)

Page 100 → [(loc: 5, ID: 3, great, 5)
, (loc: 5, ID: 2, bad, 1)]

Page 101 → [(—)]

Page 102 → [(loc —)]

Trade Offs → Data Duplication

Page No. _____

Date _____

Day _____

↳ Avg Review / location = 1

↳ Storage calculation = $500M \times 525B$
 $= 500 \times 10^6 \times 525$
 $= 262.5 GB$

Over 5 years

262.5GB $\longrightarrow$ 651 GB

Location Photos

PhotoID : 8B	$\Rightarrow$ 532B / row
locationID : 8B	
PhotoURL : 256B	
Caption : 256B	
Update at : 4B	

↳

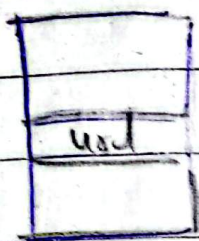
Retrieve Photos for a location X

↳ Avg Photo / Location = 5

↳ Storage Calc = $500M \times 532B \times 5$
 $= 1.3TB \longrightarrow 5 \text{ yrs} \longrightarrow 3.3TB$

Date _____

Day _____



dB



S3

cost on

↳ Storage size

↳ Data Transfer

↳ Download

Photo Storage

↳ Photo size : 750 KB

↳ 5 photos / loc :

↳ 500M loc

$$\text{Storage} = 500M \times 750 \text{ KB} \times 5$$

$$= 1.7 \text{ PB} \longrightarrow 4.3 \text{ PB Storage}$$

$$\text{For a } 200 \text{ KB} = 500 \text{ MB} \times 200 \text{ KB} \times 5$$

$$= 500 \times 10^6 \times 1000 \times 10^3$$

$$= 500 \text{ TB} \longrightarrow 5 \text{ years} \longrightarrow$$

Page No. _____

Date _____

Day _____

Step #02 : API Design

GET — /search_location

↳ Query params

↳ search_location : (long, lat)

↳ radius_of_search : number

↳ category : ENUM

↳ sort_by_filter : ENUM

↳ page : number

↳ page_size : number

→ READ INTENSIVE System

→ Vendors → new locations

500M ~~937~~ → 620M

↳ new locations = 120M → write queries
= 120M/yr (assuming no failure)

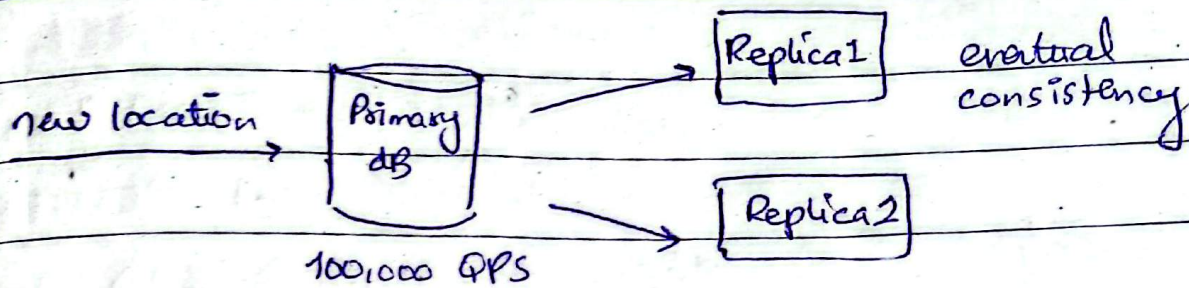
$$QPS = \frac{120M/yr}{365 \times 24 \times 60 \times 60} \approx \boxed{14QPS}$$

$$365 \times 24 \times 60 \times 60$$

An no-sql ~~has~~ cluster key = sort-key

↳ range queries are optimized acc to sort-keys.
Day

Date



add a

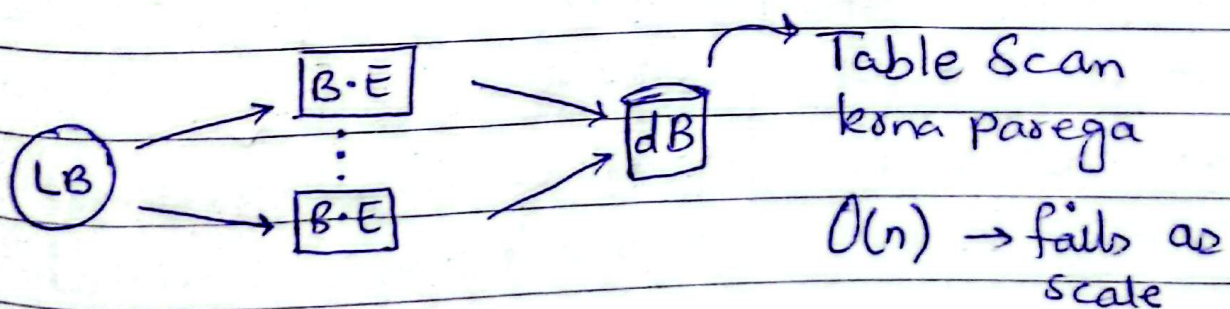
↳ Review process of 24 - 72 hrs → This time period will eventually make dB consistent
*D

→ q/ we have dynamic locations [Careem]

↳ the system would become WRITE intensive
↳ Each car will be updating its location realtime

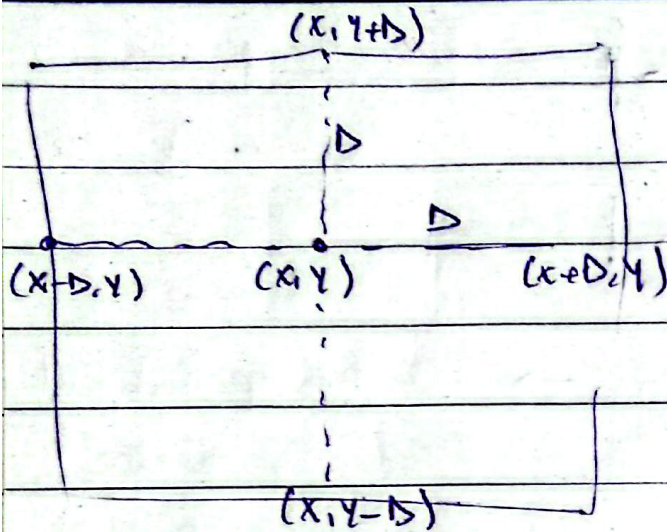
Step #03 : System Design

① Approach #01 : FYP Solution



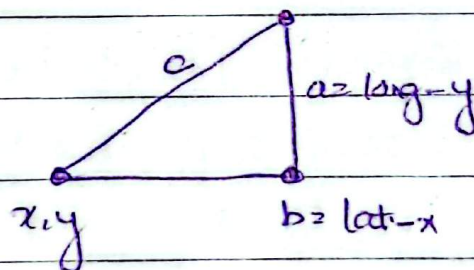
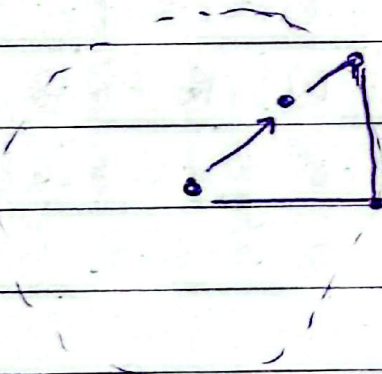
Date _____

Day _____



SELECT * FROM locations
WHERE lat BETWEEN
 $x-D$ AND $x+D$
AND
longitude BETWEEN
 $y-D$ AND $y+D$

Now For circle,



$$c^2 = a^2 + b^2$$

$$c = \sqrt{a^2 + b^2}$$

$$= \sqrt{(\text{lat} - x)^2 + (\text{long} - y)^2}$$

→ PROs

- ↳ Simple (cost)
- ↳ Low Data Vol
- ↳ 1000-10,000

→ CONS

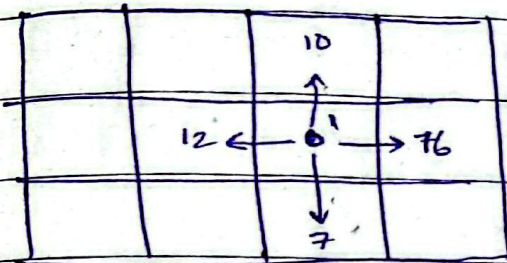
- ↳ Table Scan
- ↳ Random I/O
- ↳ Always covers straight line but the Globe is curved.

< search distance

Date _____

Day _____

① Approach #02 : Divide Map into Grid



→ divide map into grids

→ each loc will have a grid_id

→ Grid size ↑ search space

increases
↳ decrease in grid size gives less no. of results

① Find Grid_id for (x,y)

② Find neighbours for grid_id "n"

③ Now search only in the neighbours

↓
more searches
would be
required like
10 searches for
a single query

SELECT * FROM

Locations Pythagoras Theorem

AND grid_id IN [1,7,16,12,10]

→ CONs :

↳ Grid size changes with time

↳ So we would need to update grid-id on all 500M rows. (Again Table Scan)

→ Summary :

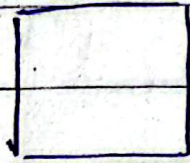
↳ This problem cannot be solved by storing & searching in dB

Page No. _____

Date _____

Day _____

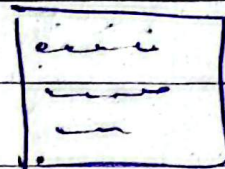
(iii) Approach #03 : Offload to Better Service/dB



grid-id

key

:



location_ids []

:

value

↳ in-memory key value dB (Redis)

→ Earth Surface $\approx 200 \text{ M miles}^2$

→ Grid size $\approx 10 \text{ miles}^2$

$$\# \text{ of grids (keys)} = \frac{200 \text{ M}}{10} > 20 \text{ M}$$

20 M grids → locations []
500 M

location ID $\approx 8 \text{ B}$

Grid ID $\approx 4 \text{ B}$

$$500 \text{ M locations IDs} \approx 500 \text{ M} \times 8 \text{ B}$$

$$(\text{memory storage}) \approx 4 \text{ GB}$$

$$\text{with long, lat} \approx 500 \text{ M} \times \overset{\text{7d long lat}}{(8+8+8)}$$

$$\approx 12 \text{ GB}$$

Page No. _____

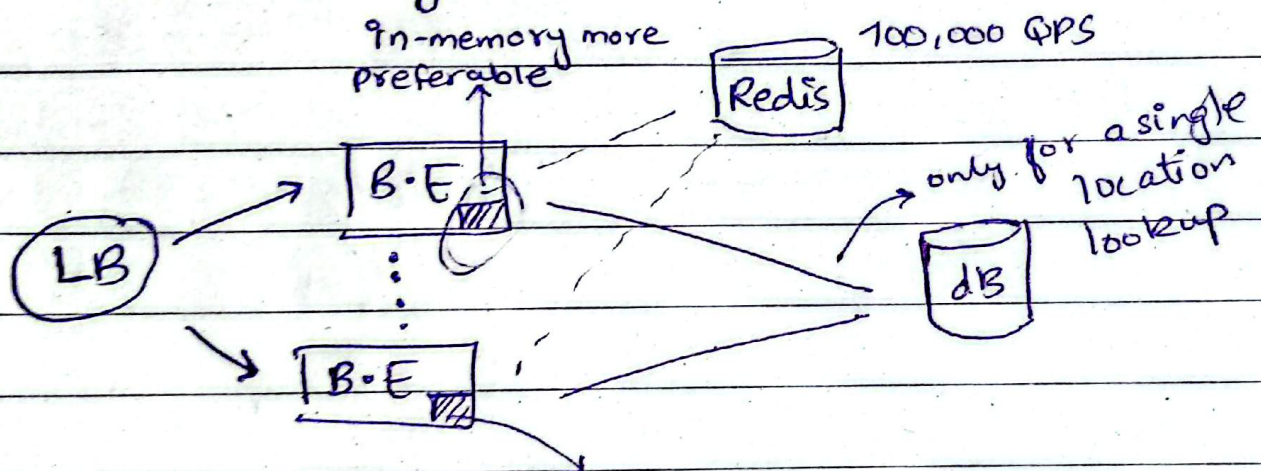
Date _____

Day _____

20M GridIDs = 20M x 4
(mem storage) = 80MB

12 GB → Search perform

↳ CPU + Memory >>> faster than dB



Cons :

↳ Cost

↳ Data Sync

↳ Replicas

↳ Instead of Redis we can also use in-memory in servers.

↳ In-memory needs to be scaled vertically by the as volume increase

• ~~Cost~~ Cost Problem:

$$\begin{aligned} \# \text{ of Cores} &= \text{QPS} \times \text{Resp time} \times \text{Safety Factor} \\ &= 100,000 \times 150\text{ms} \times 1.5 \\ &= 22500 \text{ Cores} \end{aligned}$$

reducing this reduces cost

→ 8 core machine
2810 servers

→ 4 core machine
5620 servers

Page No. _____

→ response time directly proportional to no. of cores
↳ response time increases wait time for other apis so
need to offload them to other cores.

Date _____ Day _____

→ Database Scalability

↳ still 100,000 QPS will hit the DB, so we

↳ 100,000 QPS → too much for a single DB

↳ Max a db can handle about 5000 - 10,000 QPS

of Replicas

$$\frac{100,000}{5000} = 100 \text{ Rep}$$

Date _____

Day _____

Booking.com

Search Engine - Continue...

• Problems

#1 : Size Of Grid

↳ the size must be fixed bcz we have to precompute the locations inside the grid

↳ Too small:

↳ no. of locations very less or empty

↳ Search radius needs to be increased for searching on neighbouring grids

↳ Too large:

↳ So many locations inside one grid (HOTSPOT)

#2 : Hotspot

grid_hotspot = [- - - - - → 400M] in this grid
grid_normal = [- - - - - 10K]

↗ 400M comparison to find nearest loc

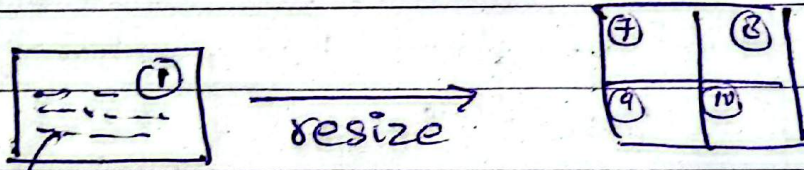
Date _____

Day _____

#03: Majority Empty Grids

↳ Most of the grids would be empty bcz of water area dominance compared to land

#04: Grid Resize



↳ prev location were referencing (1)

↳ Now they need to refer (7), - (10)

↳ I/O operation ↑

↳ remap all grid IDs

#05: Memory Growth → 20%.

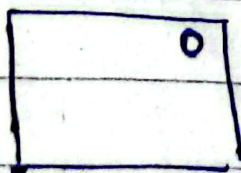
12 GB → 25 GB
5 yrs

(Backend needs to be updated or Redis)

(iv) Approach #04: Algorithms

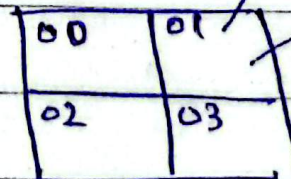
① Geo Hashing

↳

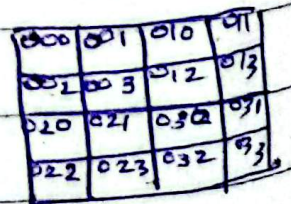


Level 0

→



→



Whole Earth

Page No. _____

Date _____

Day _____

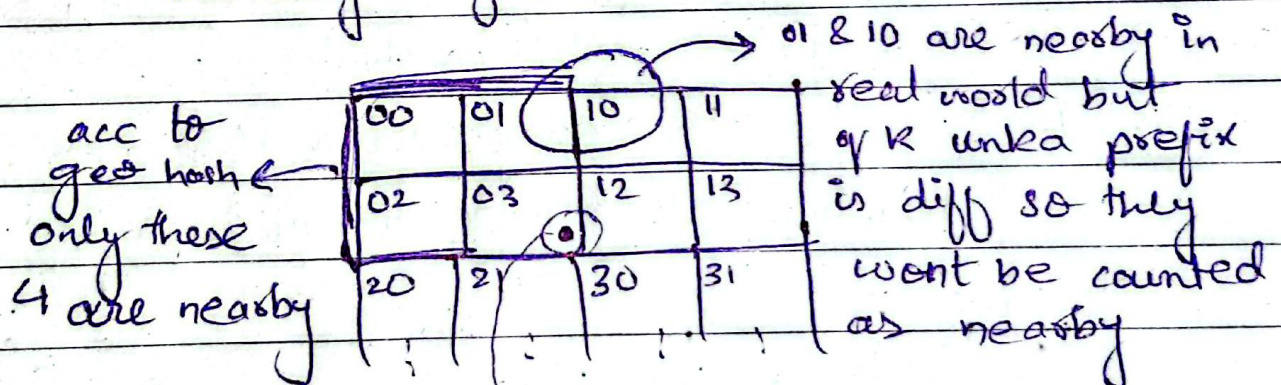
→ prefix hamesha same rhta hai.

→ Easy to find locations of same prefix

↳ SELECT * FROM
Locations where
Grid-id LIKE "010%"
prefix

Issue:

↳ EK grid actual mein nearby hogi ~~see~~ but
according to geo hash it would be not.



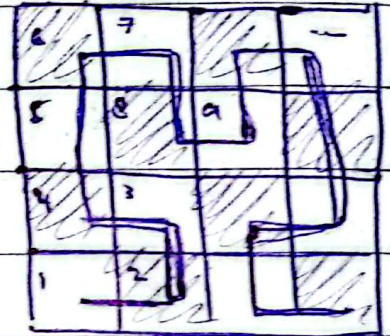
if we want to get nearby for this we would need to search in 03, 12, 30
but acc to geo hash nearby would only be
02, 03, 01, 00.

Date _____

Day _____

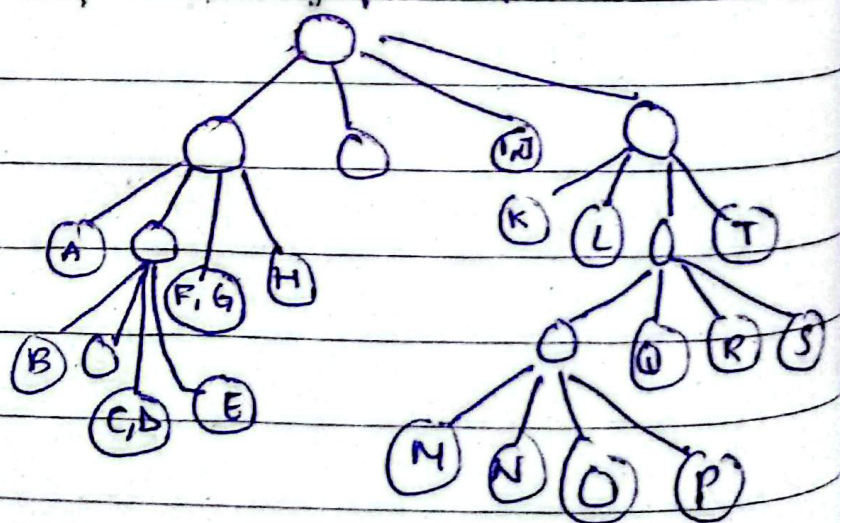
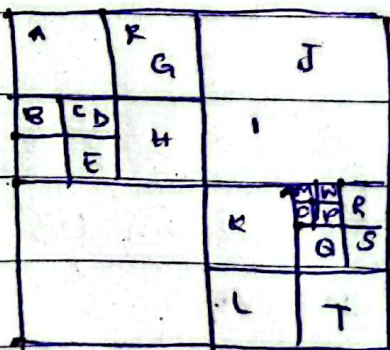
② Google S2 Library

- ↳ Maps the world into a single straight line
- ↳ This is called Hilbert Hilbert Curves
- ↳ Used in google maps.



③ Quad Trees

- ① Start with a single root node (full map)
 - ↳ Start filling in the locations
- ② As soon as the no. of loc > 500
 - ↳ Divide that into 4 equal parts



↳ Gives dynamic grids

Date _____

Day _____

If total records are of 500M ,

Search time $O(\log_2 N)$

↳ search μsec

↳ Approx 15-16 levels

• Memory Calculations

location ID	= 8B
longitude	= 8B
latitude	= 8B

Total locations = 500M

Threshold = 500

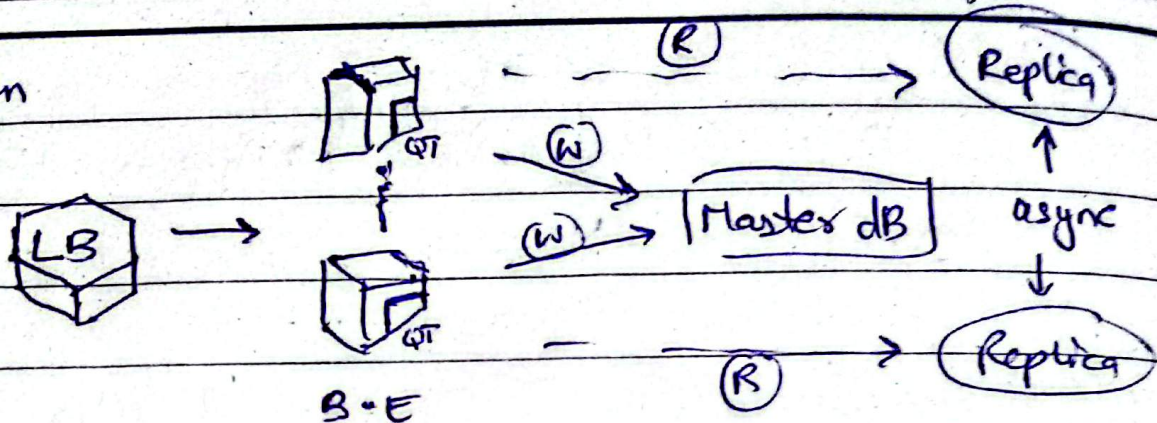
Total Grids = $\frac{500M}{500} = 1M$ grids

↳ total size = $500M \times (8B + 8B + 8B)$
= 112GB

Date _____

Day _____

→ Design



of Cores

Best case : $1 \mu s$

$$= 100,000 \times 1 \mu s \times 1.5$$

$$= 15 \text{ core} \rightarrow 3-4 \text{ servers}$$

Avg case : 1 ms

$$= 100,000 \times 1 \text{ ms} \times 1.5$$

$$= 150 \text{ cores}$$

Worst case : 10 ms

$$= 100,000 \times 10 \text{ ms} \times 1.5$$

$$= 1500 \text{ cores}$$

Issue: Memory growth

at ~~s~~ after 8 year memory requirement becomes
25 GB . Very expensive to have each
server of 25 GB

Page No. _____

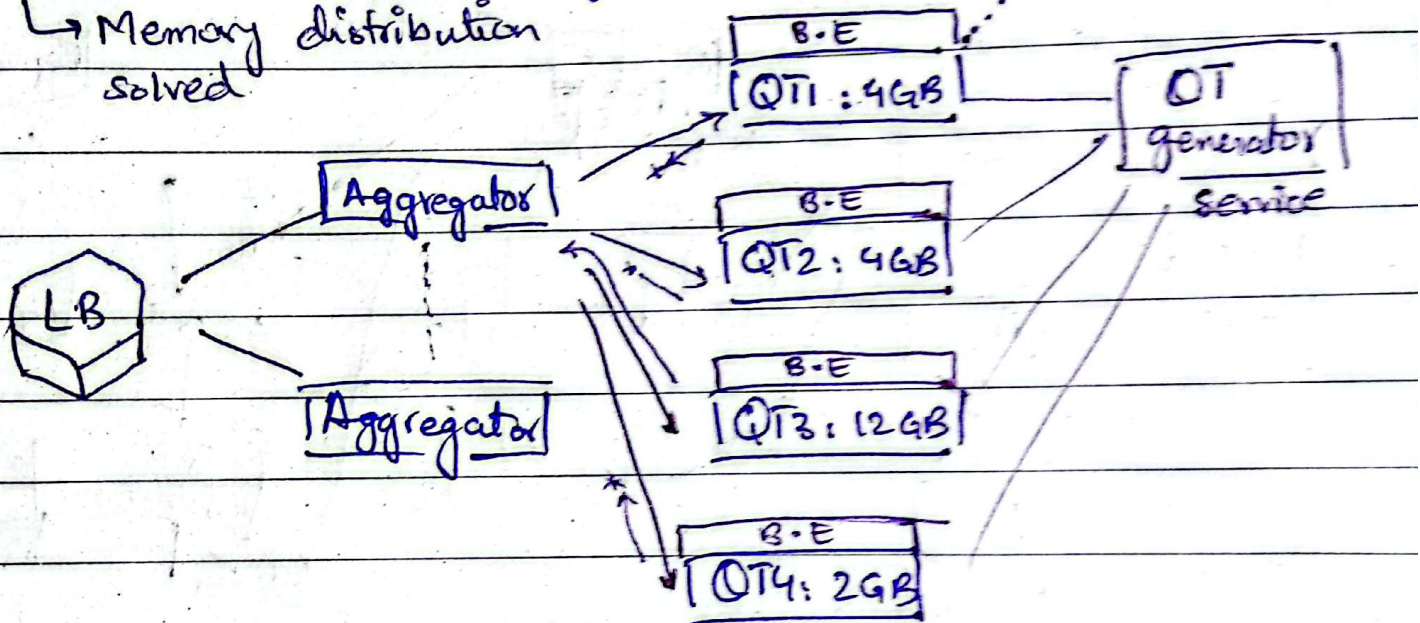
Date _____

Day _____

⑤ Approach # 05 : Memory scale by 20%.

↳ We use approach of Parallelism

↳ Memory distribution solved



↳ The aggregator queries all the Quadtrees parallelly & each one of the tree does it's own computation and sends the location if it exists

↳ QT Generator :

↳ A backend service that stores Key value pairs.

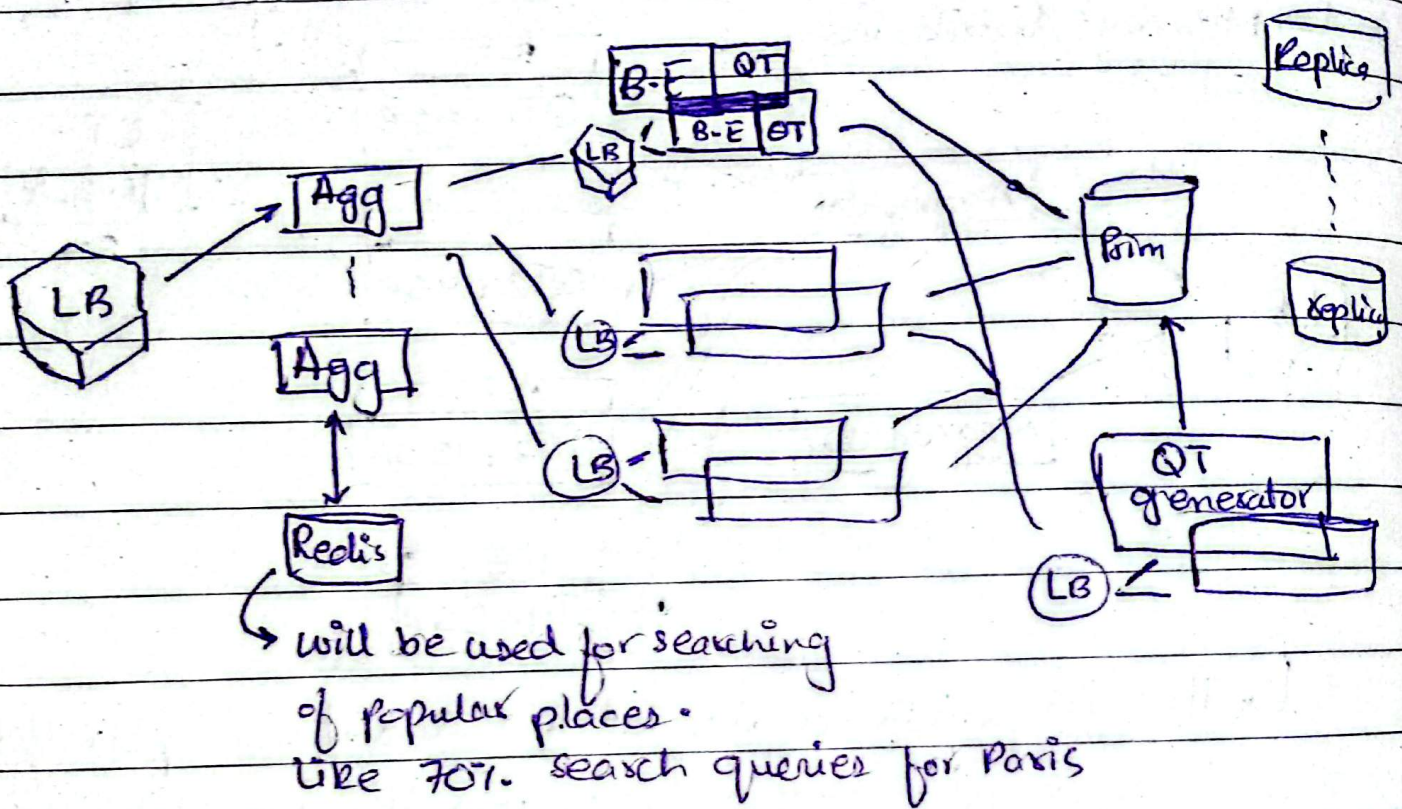
Server_1 : [loc1, loc2 -- locn]

Server_n : [—]

Date _____

Day _____

• Highly Available & Fault Tolerance Design



Q: What will happen when new loc added?

↳ A new location is added and a time period of 24 hrs - 72 hrs is given to sync

• Idempotency key

↳ System should not create duplicate resources for duplicated requests

↳ Ex: User clicks "pay now" btn twice, but payment should be deducted only once.

Page No. _____